NTNU
Kunnskap for en bedre verden

Department of Computer Science

IDIG4120 - Digital Information Processing

Project assignment 1 - Signal audit

Authors:
Katharina Kivle, Nikolai O. Stubø

September, 2026

Contents

List of Figures i

List of Tables i

1 Introduction 1

1.1 Task Description 1

1.2 The dataset 1

2 Dataset audit 2

2.1 Implementation concern 2

3 Experiments 3

3.1 Fourier Experiment 3

3.2 Quantization Experiment 6

4 Contributions 7

List of Figures

1 Results from the Fourier pruning experiment at different thresholds. 5

List of Tables

1 Dataset overview 2

2 Representation and sampling properties of the dataset. 3

3 Results from removing weak Fourier coefficients in fold1. 4

4 RMSE by subtype in fold1 after conversion 6

1 Introduction

This document contains our submission for the first assignment in the course *IDIG4120 - Digital Information Processing*. In it we will fulfill the requirements set by the task description

The purpose of this assignment is to investigate and analyze a real-world dataset, and to gain a better understanding of how digital audio is represented and processed. In our chosen dataset, we have WAV recordings of different sounds from urban environments.

Throughout the project, we examine the structure and properties of the dataset, including file formats, data types bit depths, sample rates and file sizes. We also preformed experiments on the audio signals to see how different transformations affect the representation of the sound.

The report is organised into a dataset audit, followed by the experiments and their result. The accompanying notebook contains the code used to load, analyse and process the dataset.

1.1 Task Description

The assignment asks for two things: *One runnable notebook* (not contained in this document) and *a two-page PDF or rendered notebook summary.* Additionally it asks for the following:

1. Format, array shape, data type, units, range and byte size.
2. Sampling rate or spacing. Say so where the concept does not apply, or is unknown.
3. One time or spatial view and one frequency view, both with labelled axes.
4. One transform or filtering experiment, with the prediction you made before running it.
5. One resampling or quantisation experiment that finds an artefact or a limitation.
6. A reproducible loader, and the baseline representation P2 will be measured against.
7. One Visual Informatics concern and one implementation, integrity or security concern that meet in this dataset.
8. Who did what, and the generative-AI declaration.

We have chosen to split these into separate parts, with the data loader and analysis of the dataset itself (1, 2, 3, and 6) being part of our initial audit in section 2. The frequency and quantisation experiments along with the visual informatics concern are discussed in section 3. Finally we will discuss who did what along with AI usage in section 4.

1.2 The dataset

Our chosen dataset is *UrbanSound8k* a dataset consisting of short sound samples in `.wav` format of various noises from an urban environment.

Dataset details:

Name: UrbanSound8k

Stable source: Kaggle Link: https://www.kaggle.com/datasets/chrisfilo/urbansound8k

Retrieval date: August 25th, 2026

Licence: CC BY-NC 4.0 on Kaggle; non-commercial

Scale: 8,732 clips at ≤ 4 seconds each with 10 different labels split into 10 folds

Classes/labels: *air_conditioner, car_horn, children_playing, dog_bark, drilling, engine_idling, gun_shot, jackhammer, siren, and street_music*

File formats: 8732 `.wav` files and 1 `.csv` with metadata and labels

Representation: Waveform data, various configurations (source: https://audeering.github.io/audbcards/audb-public/urbansound8k.html)

Bitdepths: 4,8,16,24,32

Channels: 1,2

Sampling rate: 8000,11024,11025,16000,22050,24000,32000,44100,48000,96000,192000

Quality risks: Varying recording quality

Integrity risks: Corrupt/malformed files, faulty/incorrect headers

Security risks: N/A

2 Dataset audit

In this section we will cover our findings from our in-depth look at various aspects of the dataset such as; `dtypes, filesizes, frame counts, wav subtypes, playtime` and more.

Property	Value
Number of WAV files	8,732
Single channel files	739
Average Filesize	935,620 Bytes
Total Size	8,169,835,940 Bytes
Average Playtime	3.6 seconds
Total Playtime	31,500.9 seconds
Average Bit Depth	21.3 Bits
Average Sample Rate	48,457 Samples per Second
Code range	−2147483648 to 2147483392

Table 1: Dataset overview

The following tables show the differences in coding and sample rates among the different files. It should be noted that the `int24` files are loaded as `int32` arrays due to how SciPy's WAV loader handles 24-bit encodings.

2.1 Implementation concern

One implementation concern is the variation in WAV representations across the dataset. SciPy, for example, loads 24-bit PCM into `int32` arrays, while unsigned 8-bit PCM uses 128 as its zero-amplitude value.

If these differences are handled incorrectly, the audit can report misleading bit depths or introduce processing errors. We therefore keep the source WAV format separate from the decoded datatype and normalize samples before comparing or transforming them.

(a) Datatypes

Datatype	Filecount
float32	169
int16	5,758
*int24	2,753
uint8	43

(b) WAV subtypes

WAV coding	Filecount
FLOAT	169
IMA_ADPCM	1
MS_ADPCM	8
PCM_16	5,758
PCM_24	2,753
PCM_U8	43

(c) Bit depths

Bit Depth	Filecount
8	43
16	5,767
32	2,922

(d) Sample rates

Sample rate (Hz)	Filecount
8,000	12
11,024	7
11,025	39
16,000	45
22,050	44
24,000	82
32,000	4
44,100	5,370
48,000	2,502
96,000	610
192,000	17

Table 2: Representation and sampling properties of the dataset.

3 Experiments

3.1 Fourier Experiment

The idea behind this experiment is to apply a Fourier transform to the sound files. We want to test whether the audio contains Fourier components that are so weak that we can throw them away to save storage space.

Prediction: We predicted that the files would take less space overall, while the most important information would remain so that the files would still be recognisable after the transformation.

What happened: For each WAV file in fold1, the experiment converted the audio waveform from the time domain into the frequency domain using a Fourier transform. In the frequency domain, the sound is represented by Fourier coefficients. Large coefficients correspond to frequencies that contribute strongly to the signal, while very small coefficients contribute relatively little.

After removing the weak coefficients, we stored only the remaining coefficients and their positions. We then reconstructed the waveform using the inverse Fourier transform and measured the difference from the original using RMSE. This tests an important idea behind compression: not all information in a signal contributes equally.

An uncompressed WAV stores every sample regardless of how important it is. In the Fourier representation, we can see that much of the signal is represented by relatively weak components. If those components can be removed without introducing too much error, the audio can potentially be represented using much less data.

There is, however, a trade-off. With a higher threshold there are fewer coefficients. This gives a smaller representation, but also more reconstruction error.

This experiment showed us that simply converting audio to Fourier coefficients is not compression by itself. As seen in the table, at a threshold of 0% the sparse representation becomes larger than

the original WAV. We see that 769.76 MiB increased to 1226.69 MiB. This happens because we must store complex Fourier values as well as their indices. Compression only becomes useful once enough weak coefficients are discarded.

There is a very strong reduction in the number of coefficients even at relatively small thresholds. At 0.1%, only 42.34% of the Fourier coefficients remain, yet the reconstruction error is only 0.001115. The stored representation is already about 31% smaller than the original WAV data.

More aggressive pruning produces very large reductions. At 5%, only 4.35% of the coefficients remain and the representation is about 92.5% smaller. However, the RMSE has risen to 0.0286, indicating that substantially more information has been lost.

Why does the strange increase happen at 0%? We are essentially not removing anything. Instead of getting a reduction, the representation increases by a little over 59%. This is explained by the fact that for every retained Fourier component, the index (`uint32`) is 4 bytes and the coefficient (`complex64`) is 8 bytes. Here we have 12 bytes per retained coefficient. The coefficient stores both the real and imaginary components, which together contain the amplitude and phase information.

The Sparse MiB value is the amount of storage required for the experimental result. The indices are the positions of the coefficients kept, and the values are the Fourier coefficients kept. There is also a small amount of information needed to reconstruct the signal. These values are then put into an array. So after discarding all Fourier coefficients weaker than 1% of the strongest component, storing the remaining coefficient values and their positions requires approximately 199 MiB.

This does not mean that the reconstructed WAV is 199 MiB. If we rewrote the reconstructed signal back to the original format, it would be roughly the original size.

It becomes smaller because once we remove enough coefficients, we no longer need to store all the index-value pairs.

Table 3: Results from removing weak Fourier coefficients in fold1.

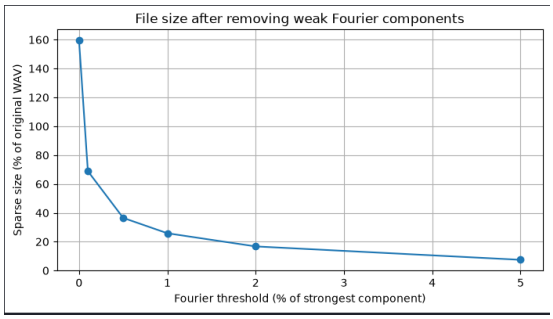
Threshold	Kept (%)	Original MiB	Sparse MiB	Reduction (%)	Mean RMSE
0.0%	100.00	769.76	1226.69	-59.36	0.000000
0.1%	42.34	769.76	530.32	31.11	0.001115
0.5%	22.17	769.76	281.05	63.49	0.004980
1.0%	15.55	769.76	199.13	74.13	0.008920
2.0%	9.95	769.76	128.90	83.25	0.015431
5.0%	4.35	769.76	57.44	92.54	0.028621

The three graphs make the trade-off easier to see. The first graph shows the file size after removing weak Fourier components. At 0%, the sparse Fourier representation is actually larger than the original WAV data because no coefficients are removed and we also have to store their indices. After increasing the threshold, the size drops quickly. At 1%, the representation is only 199.13 MiB compared to the original 769.76 MiB, and at 5% it has dropped to 57.44 MiB.

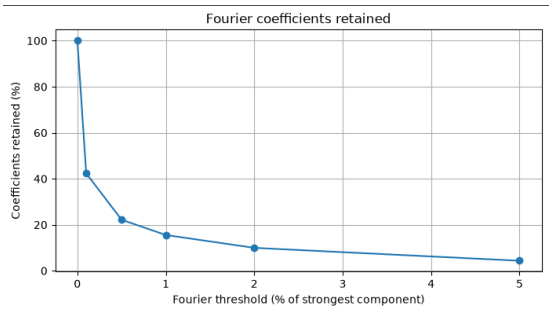
The second graph shows how many Fourier coefficients are retained. We can see that even a small threshold removes a large number of coefficients. At 0.1%, only 42.34% of the coefficients remain, and at 1% only 15.55% remain. This shows that a large part of the Fourier representation consists of relatively weak components.

The third graph shows the reconstruction error. This graph goes in the opposite direction from the first two. As we remove more coefficients, the RMSE increases. At 0% the error is zero because nothing has been removed. At 0.1% the error is still very small at 0.001115, while at 5% it has increased to 0.028621. This shows the trade-off in the experiment: removing more coefficients gives us a smaller representation, but also increases the difference between the reconstructed signal and the original.

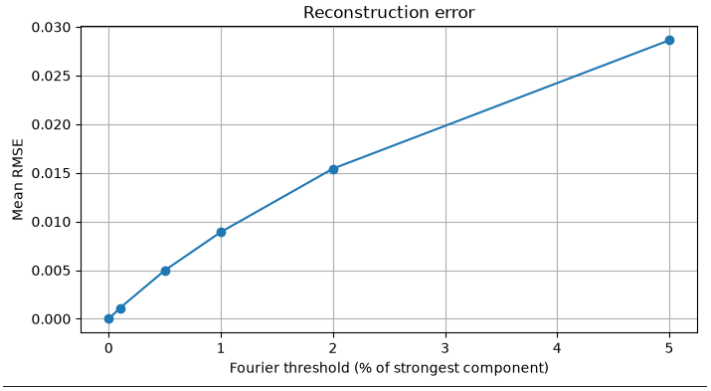
We also listened to a small selection of the reconstructed audio files. This gave us an indication of whether the increasing RMSE corresponded to a noticeable change in the sound. One thing we noticed while listening to the reconstructed files is an artifact introduced by the pruning. When



(a) File size after removing weak Fourier components.



(b) Percentage of Fourier coefficients retained.



(c) Reconstruction error measured with RMSE.

Figure 1: Results from the Fourier pruning experiment at different thresholds.

applying the threshold, the Fourier coefficient is either kept or removed completely. This makes the reconstructed signal lose some of the frequency components from the original. When the remaining components are combined again, it appears to produce a muffled, static-like noise. The removed Fourier coefficients may represent details, like background sounds. The hard thresholding itself can also produce audible artifacts because the frequency representation has been changed so abruptly. Even if the numerical error is fairly small, the error may be concentrated in frequencies that are noticeable to the ear.

We also made a script to listen to the difference between the original and the reconstructed audio. The difference is mostly static or a low hissing sound. This seems to indicate that what we removed consists of low-level broadband information. There is also an indication that it removes some parts of the signal that may be useful for categorizing the file. However, this is barely noticeable. The signal is recognisable, but the audible static shows that the compression has changed the perceived quality. The static should be possible to remove with another filter, for example, a low-pass filter. However, more audio information can also be lost.

Here are two graphs of the original and the reconstructed waveform. As we can see, the two signals overlap. Visually, they don't look that different, which can explain why the reconstructed signal is still recognisable after coefficients are removed. The lost signal is much smaller than the original signal. The original reaches amplitudes close to ± 0.8, while most of the lost signal stays around ± 0.01 to ± 0.04. There are a few larger peaks. We can also see that the loss becomes stronger where there are strong sounds in the original recording. This means that the pruning is removing some of the detail from the sound as well, not only the background noise.

We can see this more clearly in the spectrogram of the lost signal. The removed information is spread across many frequencies rather than being concentrated at a single frequency (threshold 1%). There is also some low-frequency energy throughout the recording, but during the louder sound events there are vertical areas extending through a wide range of frequencies. That broadband

loss is one reason why the reconstructed sound can have a static or hiss-like quality. So the hard threshold removes the weak frequency components, and their combined absence becomes audible even though each individual component was weak. The static is a consequence of the removed information.

3.2 Quantization Experiment

The idea behind this experiment is to standardize the different sample representations by converting all files in fold1 to signed 16-bit PCM. We want to test whether this can reduce storage while keeping the difference from the original signals small.

Our chosen target format is `int16`, as most files are already stored as 16-bit PCM, while 24-bit and floating-point files can potentially be represented using less space.

Prediction

We predicted that converting the files to 16-bit samples would reduce the total size with only a small loss in precision. Files already stored as 16-bit PCM should remain unchanged, while 24-bit and floating-point files should become smaller but introduce some quantization error. Lower precision files should not gain any new information from being stored using more bits.

Side effects

In the process of carrying out this experiment, we discovered several bugs and artifacts in the already written data loader and audit. These became apparent when reviewing unexpected results from the experiment and were iteratively fixed. The main problems were incorrect conversion of unsigned 8-bit PCM, treating 24-bit PCM as 32-bit because SciPy loads it into `int32` arrays, and an error in the frequency visualization.

What happened:

For each file, samples were first converted to normalized amplitudes before being converted to `int16`. The quantized values were then converted back to the same normalized scale and compared with the original signal using RMSE.

The original 873 WAV files in fold1 occupied 769.76 MiB. After converting them to 16-bit PCM, the same signals required 604.31 MiB, giving a reduction of 21.49%. The mean RMSE across all files was 0.00000315, showing that the difference introduced by the conversion was very small.

Source format	Files	Mean RMSE	Max RMSE
FLOAT	31	0.00000881	0.00000882
MS_ADPCM	1	0.00000000	0.00000000
PCM_16	519	0.00000000	0.00000000
PCM_24	313	0.00000791	0.00000888
PCM_U8	9	0.00000000	0.00000000

Table 4: RMSE by subtype in fold1 after conversion

Limitation

The limitation of this conversion is the loss of amplitude precision. A 16-bit sample has fewer possible values than the 24-bit and floating-point samples found in the dataset. In short: The 21.49% reduction in storage comes at the cost of making the conversion lossy.

4 Contributions

The writing of the report was collaborative with both team-members contributing. We did one experiment each, with Nikolai doing the quantization experiment and Katharina the filtering (Fourier). The respective experiments sections were mainly written by the person performing them, but were discussed and explained to the other team-member.